第三章 复合类型

目录

- 3.1 字符串与切片
- 3.2 元祖
- 3.3 结构体
- 3.4 枚举
- 3.5 数组

3.1 字符串与切片

- 字符串在Rust语言中是一个相对麻烦的结构。看一个例子：

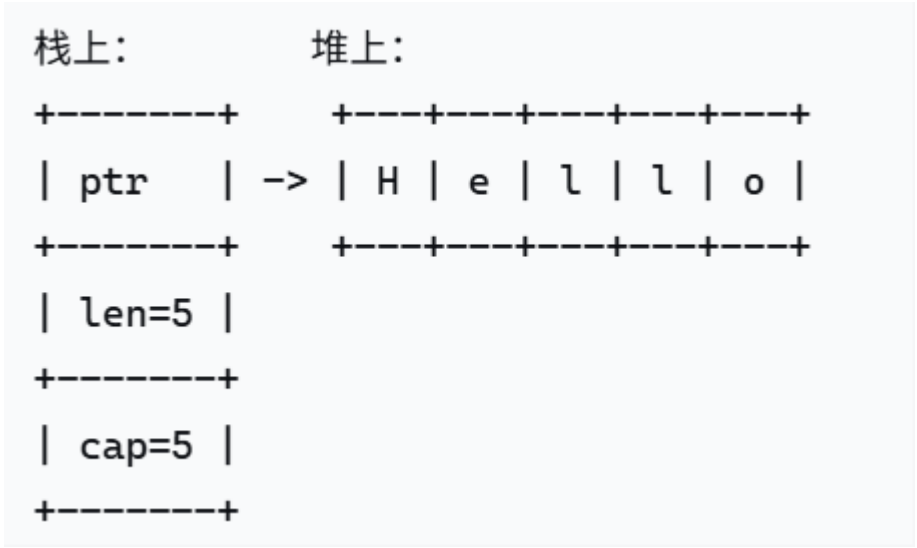
```
fn main() {
    let my_name = "Pascal";
    greet(my_name);
}

fn greet(name: String) {
    println!("Hello, {}!", name);
}
```

- 编译会报错： expected struct `std::string::String`, found `&str`
- 字符串有可变和不可变两种类型
  - C中： char [10], char *
  - C++中： string类型（类、构造函数）
  - Rust： String可变、&str不可变

3.1 字符串与切片——内存布局

String 的内存布局：



&str 的内存布局：



3.1 字符串与切片——String、&str以及char

- String类型：可变长的字符串类型，utf-8编码
  - 编译时不确定字符串长度
  - 需要显示声明，即

```
let s1 = String::new();                    // 空字符串
let s2 = String::from("hello");            // 从字面量
let s3 = "hello".to_string();              // 同上
```

- &str类型：不可变长的字符串切片类型
  - 编译时（即代码写死）已知字符串长度
  - let s = “hello world!”; 默认申明为&str
- char类型：字符类型，Unicode编码

3.1 字符串与切片——切片（slice）

- 对于字符串而言，切片就是对 String 类型中某一部分的**引用**：

```
let s = String::from("hello world");

let hello = &s[0..5];
let world = &s[6..11];
```

- hello 没有引用整个 String s，而是引用了 s 的一部分内容，通过 [0..5] 的方式来指定
- 使用方括号包括的一个左闭右开区间序列：&xx**[开始索引..终止索引]**

3.1 字符串与切片——切片（slice）

- Rust 字符串采用UTF-8 编码的字节序列。UTF-8 是一种**变长编码**:
  - ASCII 字符：1 字节
  - 常见非 ASCII 字符：2-3 字节
  - 某些字符（如表情符号）：4 字节
- 要注意不要用C语言的理解:
  - C语言中字符串是由多个字符型变量组成，而Rust不是!

怎样去理解字符串序号?

`char` 类型（4字节）

```rust
let c: char = 'A';     // 4字节存储
let e: char = '😊';    // 也是4字节存储
```

`String` / `&str`（UTF-8 编码）

```rust
let s = "A";     // 1字节: 0x41
let s = "😊";    // 4字节: 0xF0 0x9F 0x98 0x8A
```

3.1 字符串与切片——切片（slice）

- 要注意：**字符串切片索引是基于字节的，而不是字符序号**
  - 字符串切片索引必须在**有效的 UTF-8 字符边界**上

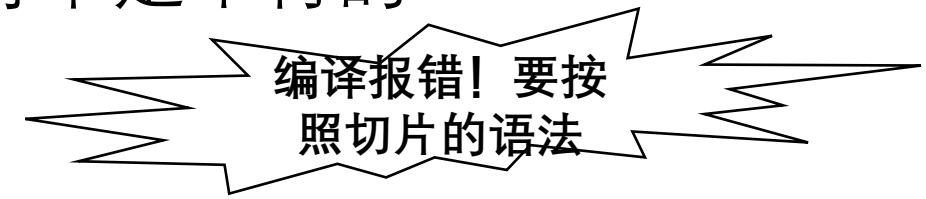
```
let s = "hello";
println!("{}", &s[0..1]); // "h" - 正确
println!("{}", &s[0..2]); // "he" - 正确

let s = "你好";
println!("{}", &s[0..3]); // "你" - 正确（中文字符占3字节）
// println!("{}", &s[0..2]); // 运行时 panic! 不是有效的字符边界
```

3.1 字符串与切片——字符串索引

- 续：用数组下标的方式去访问Rust字符串是不行的

```
let s1 = String::from("hello");
let h = s1[0];
```



  - 需要准确理解下标在C语言中的含义：每个元素长度都一样的顺序存储方式
- 不同的字符形式在utf-8中占用空间不一样
  - `let hello = String::from("Hola"); \\4字节`
  - `let hello = String::from("中国人"); \\9字节`
- 隐藏好处：和ASCII码兼容。

3.1 字符串与切片——操作字符串

- Rust 字符串的修改，添加，删除等常用方法，详细请看教材或AI
- 以push为例：

```
fn main() {
    let mut s = String::from("Hello ");

    s.push_str("rust");
    println!("追加字符串 push_str() -> {}", s);

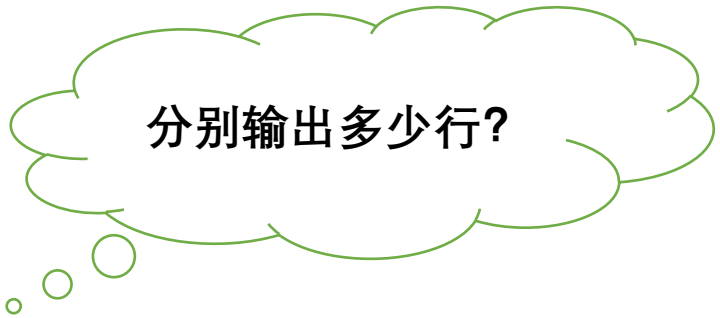
    s.push('!');
    println!("追加字符 push() -> {}", s);
}
```

- 和C++中“string类”使用方法比较像

3.1 字符串与切片——字符串的遍历

- 以字符（Unicode）方式遍历：chars()函数

```
for c in "中国人".chars() {
    println!("{}", c);
}
```



- 以字节方式遍历：bytes()函数

```
for b in "中国人".bytes() {
    println!("{}", b);
}
```

3.1 字符串与切片——String与垃圾回收

- 当一个String类型变量离开作用域时，堆中分配的内存如何处理?
- 先看C++中string是怎样处理的?

```
#include <string>
int fun (){
    std::string s = "hello";
}
```

- C++里面的string是个“类”，类都有构造函数和析构函数。类变量在销毁时自动调用析构函数释放内存
  - 存在潜在的空指针（即悬空）问题

3.1 字符串与切片——String与垃圾回收

- C++中string释放的空指针：销毁之前返回了指向字符串的指针

```
std::string* create_string() {
    std::string s = "hello";
    return &s; // 危险！返回局部变量的地址
} // s 被销毁，返回的指针悬空

const char* get_c_str() {
    std::string s = "hello";
    return s.c_str(); // 危险！c_str() 在 s 销毁后无效
}
```

3.1 字符串与切片——String与垃圾回收

- Rust基于所有权
  - **违反规则编译不通过**

```rust
fn create_string() -> String {
    let s = String::from("hello");
    s // 所有权转移，不会出现悬空指针
}

fn invalid_example() -> &String {
    let s = String::from("hello");
    &s // 编译错误！s 即将被释放，不能返回引用
}
```

- C++基于析构函数
  - **运行时才会潜在报错**

```cpp
std::string* create_string() {
    std::string s = "hello";
    return &s; // 危险！返回局部变量的地址
} // s 被销毁，返回的指针悬空

const char* get_c_str() {
    std::string s = "hello";
    return s.c_str(); // 危险！c_str() 在 s 销毁后无效
}
```

3.2 元组

- 用括号括起来的多个元素组成的复合变量，类似结构体
  - 变量数、类型在申明时即确定，两种声明方式：

```
fn main() {
    let tup: (i32, f64, u8) = (500, 6.4, 1);
}
```

```
fn main() {
    let tup = (500, 6.4, 1);

    let (x, y, z) = tup;

    println!("The value of y is: {}", y);
}
```

3.2 元组

- 用 . 来以下标方式访问元组中的元素（下标从0开始）

```
fn main() {
    let x: (i32, f64, u8) = (500, 6.4, 1);

    let five_hundred = x.0;

    let six_point_four = x.1;

    let one = x.2;
}
```

3.2 元组

- 函数返回值场景中使用元组一次返回多个值。

```
fn main() {
    let s1 = String::from("hello");

    let (s2, len) = calculate_length(s1);

    println!("The length of '{}' is {}.", s2, len);
}

fn calculate_length(s: String) -> (String, usize) {
    let length = s.len(); // len() 返回字符串的长度

    (s, length)
}
```

3.3 结构体

- 语法和C语言类似
  - 定义：

```
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}
```

- 申明结构体变量：

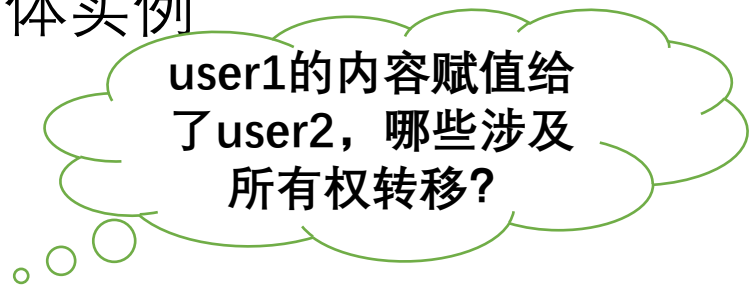
```
let mut user1 = User {
    email: String::from("someone@example.com"),
    username: String::from("someusername123"),
    active: true,
    sign_in_count: 1,
};
```

- 申明的同时进行初始化动作，并标记整个结构体是否可变

3.3 结构体

- 结构体赋值的所有权转移问题
  - 例：根据已有的结构体实例，创建新的结构体实例

```
let user2 = User {
        active: user1.active,
        username: user1.username,
        email: String::from("another@example.com"),
        sign_in_count: user1.sign_in_count,
    };
```



  - 仅user2.username的所有权发生了转移：回忆所有权转移的规则
    - 部分所有权的转移使得user1也无法再被使用。

3.4 枚举

- 语法结构和C语言类似，但表达能力更强
  - 类C功能的表达：

```
enum Color {
    Red,    // 默认从 0 开始
    Green,  // 1
    Blue,   // 2
}
```

枚举颜色，数据默认从0开始

```
// 可以显式指定判别值
enum HttpStatus {
    Ok = 200,
    NotFound = 404,
    InternalServerError = 500,
}
```

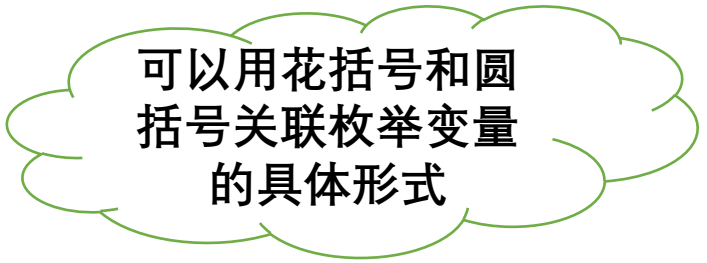
也可以显示给枚举变量一个值

3.4 枚举

- 除此之外，Rust 的枚举**可以携带不同类型和数量的数据**

```rust
// Rust - 强大的枚举
enum Message {
    Quit,                       // 无关联数据
    Move { x: i32, y: i32 },    // 包含一个匿名结构体
    Write(String),              // 包含一个 String
    ChangeColor(u8, u8, u8),    // 包含三个 u8
}

// 使用示例
let msg1 = Message::Quit;
let msg2 = Message::Move { x: 10, y: 20 };
let msg3 = Message::Write("hello".to_string());
let msg4 = Message::ChangeColor(255, 0, 0);
```



- 圆括号：关联元组
- 花括号：关联结构体
- 无：不关联数据
- **每个枚举变量都是一个构造函数！**

3.4 枚举

- 枚举相对结构体的好处：同一化类型，即**一组相关的可能值**，需要统一处理时：
  - 用枚举和结构体分别表示的例子：

```
enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}

fn main() {
    let m1 = Message::Quit;
    let m2 = Message::Move{x:1,y:1};
    let m3 = Message::ChangeColor(255,255,0);
}
```

```
struct QuitMessage; // 单元结构体
struct MoveMessage {
    x: i32,
    y: i32,
}
struct WriteMessage(String); // 元组结构体
struct ChangeColorMessage(i32, i32, i32); // 元组结构体
```

四个**逻辑意义上相近**的“消息”，如果用结构体表示会成为四个相互独立的类型

3.4 枚举——Option枚举类型

- Option 枚举用于处理空值，即C语言中的“null”

把null变成了一种类型，由编译器强制检查避免出现null异常

```c
// C语言 - 可能产生空指针解引用
char* find_name(int id) {
    if (id == 1) return "Alice";
    else return NULL;  // 可能返回空指针
}

void risky_code() {
    char* name = find_name(999);
    printf("Name length: %zu\n", strlen(name));  // 可能崩溃!
}
```

```rust
rust

fn find_name(id: i32) -> Option<&'static str> {
    if id == 1 {
        Some("Alice")
    } else {
        None  // 不会产生空指针
    }
}

fn safe_code() {
    let name = find_name(999);
    match name {
        Some(n) => println!("Name length: {}", n.len()),  // 安全访问
        None => println!("Name not found"),               // 必须处理None情况
    }
    // 编译器强制你处理所有可能情况!
}
```

3.4 枚举——Option枚举类型

- Option 枚举包含两个成员，一个成员表示含有值：Some(T), 另一个表示没有值：None，定义如下：

```
enum Option<T> {
    Some(T),  // 有值，包含具体的数据
    None,     // 无值
}
```

  - 其中 T 是泛型参数，Some(T)表示该枚举成员的数据类型是 T，换句话说，Some 可以包含任何类型的数据
  - 比如：Some(i32)可理解为存在这么一个i32类型的值且不为空

3.4 枚举——Option枚举类型

```
let some_number = Some(5);
let some_string = Some("a string");

let absent_number: Option<i32> = None;
```

- None 值跟空值具有相同的意义，即不存在。那么为什么要替换null?

3.4 枚举——Option枚举类型

- C语言版本：可以正常编译和运行

```c
int x = NULL;
int y = 10;
printf("%d", x + y);
```

- Rust语言版本：编译报错!

```rust
let x: i8 = 5;
let y: Option<i8> = Some(5);

let sum = x + y;
```

```
error[E0277]: the trait bound `i8: std::ops::Add<std::option::Option<i8>>` is
not satisfied
 -->
  |
5 |     let sum = x + y;
  |                   ^ no implementation for `i8 + std::option::Option<i8>`
  |
```

- C中NULL是一个值，编译器无法检查值；
- Rust的None是包在Option中的枚举类型，由编译器检查避免了期望某值不为空但实际上为空的情况

- Rust 不知道该如何将 Option<i8> 与 i8 相加

3.4 枚举——Option枚举类型

- 需要程序员确定哪些情况是包含空值的，存在空值情况则必须使用Option<T>类型
  - 换句话说，没有Option<T>的时候一定不会有空值情况
- 在对 Option<T> 进行 T 的运算之前必须将其转换为 T
  - 具体的转换方式目前不涉及

3.5 数组

- 两种数组结构：array和vector
  - array——数组：定义时固定长度（分配在栈上）
  - vector——动态数组：定义后还可以动态增长（分配在堆上）
- 特征：
  - 数组元素类型一致
  - 线性存储方式
- 这里默认“数组”这个词指的是array

3.5 数组——定义一维数组

- 数组的定义方法：

```rust
// 方式1：明确指定类型和长度
let arr1: [i32; 5] = [1, 2, 3, 4, 5];

// 方式2：类型推断
let arr2 = [1, 2, 3, 4, 5]; // 编译器推断为 [i32; 5]
```

- Rust 数组的类型格式是：**[T; N]**
  - T：元素类型
  - N：数组长度（编译时常量）

- 初始化相同的值：

```rust
// 创建包含 5 个 0 的数组
let zeros: [i32; 5] = [0; 5];

// 创建包含 10 个 true 的数组
let bools = [true; 10];

// 创建包含 3 个 "hello" 的数组
let strings = ["hello"; 3];
```

3.5 数组——访问

- 常规访问：和C语言一样，通过下标形式访问数组元素

```
fn main() {
    let a = [9, 8, 7, 6, 5];

    let first = a[0]; // 获取a数组第一个元素
    let second = a[1]; // 获取第二个元素
}
```

- 切片访问：引用集合中的部分连续片段

```
let a: [i32; 5] = [1, 2, 3, 4, 5];

let slice: &[i32] = &a[1..3];

assert_eq!(slice, &[2, 3]);
```

这里数组切片变量 slice 的类型是&[i32]，与之对比，数组的类型是[i32;5]

3.5 数组——越界访问

- 数组越界的处理，回忆一下数组越界的内涵以及C语言是如何处理的？

3.5 数组——越界访问，两种情况

- 编译时检查，越界报错

```rust
fn main() {
    let arr = [1, 2, 3, 4, 5];

    // 这些在编译时就能发现错误
    // let x = arr[5];  // 编译错误: index out of bounds
    // let y = arr[10]; // 编译错误: index out of bounds

    // 合法的访问
    let first = arr[0];  // OK
    let last = arr[4];   // OK
}
```

- 运行时检查，越界panic
  - 有些访问是运行时才能确定边界

```rust
fn main() {
    let arr = [1, 2, 3, 4, 5];
    let index = 10; // 运行时才确定的值

    // 这会导致运行时 panic
    // let value = arr[index]; // thread 'main' panicked
```

3.5 数组——越界访问，和C语言的差异?

- C 语言：未定义行为

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    // 越界访问 - 未定义行为
    int value = arr[10];  // 可能返回垃圾值、程序崩溃或更糟

    printf("越界值: %d\n", value); // 无法预测结果

    // 越界写入 - 可能破坏其他内存
    arr[10] = 100; // 危险！可能破坏栈帧或重要数据

    return 0;
}
```

- Rust：确定性行为

特性	C 语言	Rust
编译时检查	基本没有	对常量索引有严格检查
运行时检查	默认关闭，需要手动开启	默认开启
越界读取	返回垃圾值，继续执行	panic 或返回 Option
越界写入	静默破坏内存，安全漏洞	panic，程序中止
错误处理	无标准机制	明确的错误处理路径

3.5 数组——越界访问，和C语言的差异?

- C 语言的危险场景

```c
void buffer_overflow() {
    int arr[3] = {1, 2, 3};
    int secret = 42;

    printf("secret before: %d\n", secret);

    // 越界写入可能修改相邻变量
    arr[5] = 100;  // 未定义行为

    printf("secret after: %d\n", secret); // 可能被修改!
}
```